

Elaborato NCIS - SDN Project Work : Network Slicing with Ryu and Mininet

Davide Di Matteo - M63001865
dav.dimatteo@studenti.unina.it

Università degli studi di Napoli Federico II — 26/03/2026

Introduzione

Il progetto mira a implementare il network slicing in una rete virtualizzata mediante *Mininet*, sfruttando la libreria Python e l'emulatore di controller *Ryu*. L'implementazione è organizzata nelle seguenti sezioni:

1. **Topology-slicing:** Restrizione delle rotte di comunicazione tra specifici host;
2. **Service-slicing:** Differenziazione del trattamento di determinati tipi di traffico.

Per ogni sezione sono previsti test di validazione dei risultati ottenuti.

1 Topology slicing

Questa sezione riguarda la definizione della topologia di rete in Mininet e la conseguente suddivisione in due *slices* isolati. Lo scopo è costruire una topologia secondo la quale gli host possono comunicare attraverso rotte predefinite:

- L'host h1 può comunicare solo con h3 tramite il path superiore;
- L'host h2 può comunicare solo con h4 tramite il path inferiore.

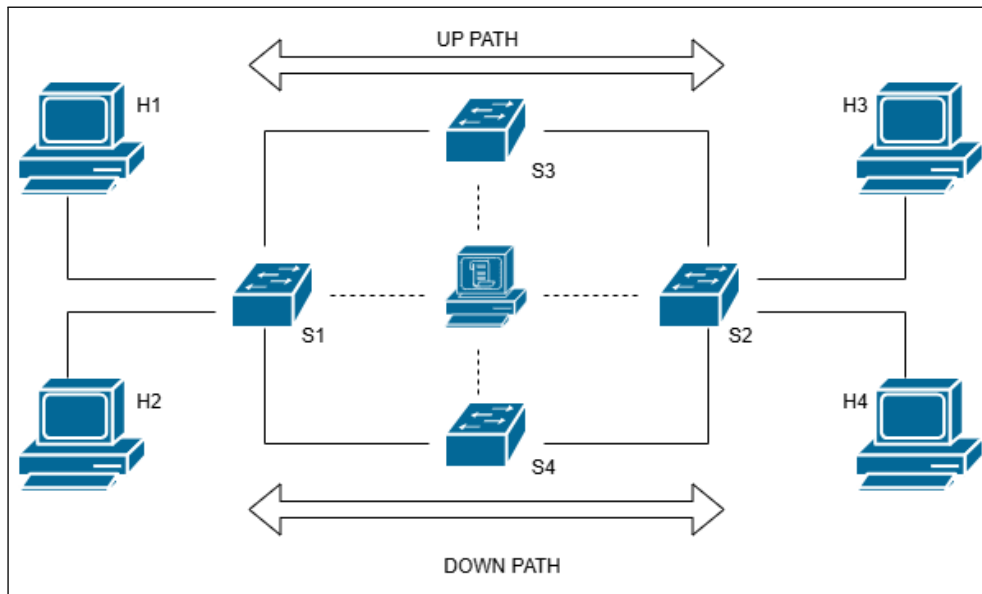
A questo proposito, è stata definita la seguente configurazione Mininet implementando il costruttore della classe `Mininet.Environment`, la cui vista logica è rappresentata in figura 1.

```
1 class Environment(object):
2     def __init__(self):
3         # ...
4         # Definizione hosts
5         self.h1 = self.net.addHost('h1', mac= '00:00:00:00:00:01', ip= '10.0.0.1')
6         self.h2 = self.net.addHost('h2', mac= '00:00:00:00:00:02', ip= '10.0.0.2')
7         self.h3 = self.net.addHost('h3', mac= '00:00:00:00:00:03', ip= '10.0.0.3')
8         self.h4 = self.net.addHost('h4', mac= '00:00:00:00:00:04', ip= '10.0.0.4')
9
10        # Definizione switch
11        self.s1 = self.net.addSwitch('s1')
12        self.s2 = self.net.addSwitch('s2')
13        self.Up = self.net.addSwitch('s3') # up
14        self.Dw = self.net.addSwitch('s4') # down
15
16        # Link declaration
17        self.net.addLink(self.h1, self.s1, delay='0.01ms', port1=1, port2=1)
```

```

18 self.net.addLink(self.h2, self.s1, delay='0.01ms', port1=1, port2=2)
19 self.net.addLink(self.h3, self.s2, delay='0.01ms', port1=1, port2=3)
20 self.net.addLink(self.h4, self.s2, delay='0.01ms', port1=1, port2=4)
21 self.net.addLink(self.s1, self.Up, bw=10, delay='0.025ms', port1=3, port2=1)
22 self.net.addLink(self.Up, self.s2, bw=10, delay='0.025ms', port1=2, port2=1)
23 self.net.addLink(self.s1, self.Dw, bw=1, delay='0.025ms', port1=4, port2=1)
24 self.net.addLink(self.Dw, self.s2, bw=1, delay='0.025ms', port1=2, port2=2)
25 # ...

```



```

1 # Estensione della classe base
2 class RyuController(app_manager.RyuApp):
3     # OpenFlow 1.3 (Rifiuta linguaggi precedenti)
4     OFP_VERSIONS = [ofproto_v1_3.OFP_VERSION]
5
6     def __init__(self, *args, **kwargs):
7         super(RyuController, self).__init__(*args, **kwargs)
8
9         self.h1 = '00:00:00:00:00:01'
10        self.h2 = '00:00:00:00:00:02'
11        self.h3 = '00:00:00:00:00:03'
12        self.h4 = '00:00:00:00:00:04'
13
14    # Installa una flow rule OpenFlow nello switch
15    def add_flow(self, datapath, priority, match, actions, idle_timeout=0, flag=0):
16        ofproto = datapath.ofproto
17        parser = datapath.ofproto_parser
18
19        inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS, actions)]
20        mod = parser.OFPFlowMod(
21            datapath=datapath, priority=priority,
22            match=match, instructions=inst,
23            idle_timeout = idle_timeout, flags=flag
24        )
25
26        # invio della regola allo switch
27        datapath.send_msg(mod)

```

Date le ridotte dimensioni della rete, sono state inserite delle regole nel controllore che impongono le rotte.



Info: RyuController è un controller SDN OpenFlow 1.3 basato su Ryu che usa Mininet e Open vSwitch. È stato implementato a partire dalla classe base `ryu.base.app_manager.RyuApp`, e gli indirizzi MAC degli host sono stati inseriti come stringhe statiche direttamente nel costruttore.

Quando Mininet avvia uno switch, questo prova ad aprire una connessione TCP verso il controller Ryu. Se la connessione viene stabilita (handshake a tre vie), viene avviato l'handshake OpenFlow. Dopodichè, lo switch invia un messaggio FEATURES_REPLY, ed il controller esegue switch_features_handler.

```

1 @set_ev_cls(ofp_event.EventOFPSwitchFeatures, CONFIG_DISPATCHER)
2 def switch_features_handler(self, ev):
3     datapath = ev.msg.datapath
4     ofproto = datapath.ofproto
5     parser = datapath.ofproto_parser
6     dpid = datapath.id
7
8     if dpid == 1:
9         # UP flow
10        port_out_Up = 3
11        port_in_Up = 1
12
13        # Flow IN
14        self.add_mac_flow(datapath, self.h1, self.h3, port_out_Up)
15        self.add_arp_flow(datapath, self.h1, port_out_Up)
16
17        # Flow Back
18        self.add_mac_flow(datapath, self.h3, self.h1, port_in_Up)
19        self.add_arp_flow(datapath, self.h3, port_in_Up)
20
21        # Down flow configuration
22        port_out_Dw = 4
23        port_in_Dw = 2
24
25        # Flow IN
26        self.add_mac_flow(datapath, self.h2, self.h4, port_out_Dw)
27        self.add_arp_flow(datapath, self.h2, port_out_Dw)
28
29        # Flow Back
30        self.add_mac_flow(datapath, self.h4, self.h2, port_in_Dw)
31        self.add_arp_flow(datapath, self.h4, port_in_Dw)
32
33    elif dpid == 2:
34        port_out_Up = 1
35        port_in_Up = 3
36
37        self.add_mac_flow(datapath, self.h3, self.h1, port_out_Up)
38        self.add_arp_flow(datapath, self.h3, port_out_Up)
39
40        self.add_mac_flow(datapath, self.h1, self.h3, port_in_Up)
41        self.add_arp_flow(datapath, self.h1, port_in_Up)
42
43        port_out_Dw = 2
44        port_in_Dw = 4
45
46        self.add_mac_flow(datapath, self.h4, self.h2, port_out_Dw)
47        self.add_arp_flow(datapath, self.h4, port_out_Dw)
48
49        self.add_mac_flow(datapath, self.h2, self.h4, port_in_Dw)
50        self.add_arp_flow(datapath, self.h2, port_in_Dw)
51
52    elif dpid == 3:
53        port_1 = 1
54        port_2 = 2
55
56        self.add_mac_flow(datapath, self.h1, self.h3, port_2)
57        self.add_arp_flow(datapath, self.h1, port_2)
58
59        self.add_mac_flow(datapath, self.h3, self.h1, port_1)
60        self.add_arp_flow(datapath, self.h3, port_1)
61
62    elif dpid == 4:
63        port_1 = 1
64        port_2 = 2
65
66        self.add_mac_flow(datapath, self.h2, self.h4, port_2)
67        self.add_arp_flow(datapath, self.h2, port_2)
68
69        self.add_mac_flow(datapath, self.h4, self.h2, port_1)
70        self.add_arp_flow(datapath, self.h4, port_1)

```

Dopo aver determinato l'identità dello switch con *dpid*, il controller crea la regola attraverso *add_mac_flow*, ovvero l'associazione (mac_src, mac_dst) → port. Attraverso la regola *add_arp_flow*, imponiamo che i messaggi ARP possono uscire su più porte, ed è una regola a priorità più alta di quella che impone il flusso basato su MAC. Osserviamo che comunque il pacchetto ARP, che viene inviato in broadcast, viene comunque confinato nello *slice* della rete.

```

1  def add_mac_flow(self, datapath, src, dst, out_port, priority=10):
2      parser = datapath.ofproto_parser
3
4      match = parser.OFPMatch(eth_src=src, eth_dst=dst)
5      actions = [parser.OFPActionOutput(out_port)]
6      self.add_flow(datapath, priority, match, actions)
7
8  def add_arp_flow(self, datapath, eth_src, out_port):
9      parser = datapath.ofproto_parser
10
11     match = parser.OFPMatch(eth_src=eth_src, eth_dst="ff:ff:ff:ff:ff:ff", eth_type=0
12                             x0806)
13     actions = []
14     if not isinstance(out_port, list):
15         out_port = [out_port]
16
17     for port in out_port:
18         actions.append(parser.OFPActionOutput(port))
19     self.add_flow(datapath, priority=20, match=match, actions=actions)

```

Per testare questa configurazione sono stati usati i tool *pingall* di Mininet e *Whreshark*.

Command Line

```

mininet> pingall
*** Ping: testing ping reachability

h1 -> X h3 X
h2 -> X X h4
h3 -> h1 X X
h4 -> X h2 X

*** Results: 66% dropped (4/12 received)

```

00:00:00_00:00:01	00:00:00_00:00:03	ARP	42 Who has 10.0.0.3? Tell 10.0.0.1
00:00:00_00:00:03	00:00:00_00:00:01	ARP	42 Who has 10.0.0.1? Tell 10.0.0.3
00:00:00_00:00:01	00:00:00_00:00:03	ARP	42 10.0.0.1 is at 00:00:00:00:00:01
00:00:00_00:00:03	00:00:00_00:00:01	ARP	42 Who has 10.0.0.1? Tell 10.0.0.3
00:00:00_00:00:01	00:00:00_00:00:03	ARP	42 Who has 10.0.0.3? Tell 10.0.0.1
00:00:00_00:00:03	00:00:00_00:00:01	ARP	42 10.0.0.3 is at 00:00:00:00:00:03
00:00:00_00:00:01	00:00:00_00:00:03	ARP	42 10.0.0.1 is at 00:00:00:00:00:01
00:00:00_00:00:03	00:00:00_00:00:01	ARP	42 10.0.0.3 is at 00:00:00:00:00:03
10.0.0.1	10.0.0.3	ICMP	98 Echo (ping) request id=0xab4c, seq=7/1792, ttl=64 (reply in 34)
10.0.0.3	10.0.0.1	ICMP	98 Echo (ping) reply id=0xab4c, seq=7/1792, ttl=64 (request in 33)
10.0.0.1	10.0.0.3	ICMP	98 Echo (ping) request id=0xab4c, seq=7/1792, ttl=64 (reply in 36)
10.0.0.3	10.0.0.1	ICMP	98 Echo (ping) reply id=0xab4c, seq=7/1792, ttl=64 (request in 35)
10.0.0.1	10.0.0.3	ICMP	98 Echo (ping) request id=0xab4c, seq=8/2048, ttl=64 (reply in 38)
10.0.0.3	10.0.0.1	ICMP	98 Echo (ping) reply id=0xab4c, seq=8/2048, ttl=64 (request in 37)

00:00:00_00:00:02	00:00:00_00:00:04	ARP	42 Who has 10.0.0.4? Tell 10.0.0.2
00:00:00_00:00:04	00:00:00_00:00:02	ARP	42 Who has 10.0.0.2? Tell 10.0.0.4
00:00:00_00:00:02	00:00:00_00:00:04	ARP	42 10.0.0.2 is at 00:00:00:00:00:02
00:00:00_00:00:04	00:00:00_00:00:02	ARP	42 10.0.0.4 is at 00:00:00:00:00:04
00:00:00_00:00:02	00:00:00_00:00:04	ARP	42 Who has 10.0.0.4? Tell 10.0.0.2
00:00:00_00:00:04	00:00:00_00:00:02	ARP	42 Who has 10.0.0.2? Tell 10.0.0.4
00:00:00_00:00:02	00:00:00_00:00:04	ARP	42 10.0.0.4 is at 00:00:00:00:00:02
00:00:00_00:00:04	00:00:00_00:00:02	ARP	42 10.0.0.2 is at 00:00:00:00:00:04
10.0.0.2	10.0.0.4	ICMP	98 Echo (ping) request id=0x13d9, seq=7/1792, ttl=64 (reply in 34)
10.0.0.4	10.0.0.2	ICMP	98 Echo (ping) reply id=0x13d9, seq=7/1792, ttl=64 (request in 33)
10.0.0.2	10.0.0.4	ICMP	98 Echo (ping) request id=0x13d9, seq=7/1792, ttl=64 (reply in 36)
10.0.0.4	10.0.0.2	ICMP	98 Echo (ping) reply id=0x13d9, seq=7/1792, ttl=64 (request in 35)
10.0.0.2	10.0.0.4	ICMP	98 Echo (ping) request id=0x13d9, seq=8/2048, ttl=64 (reply in 38)
10.0.0.4	10.0.0.2	ICMP	98 Echo (ping) reply id=0x13d9, seq=8/2048, ttl=64 (request in 37)

2 Service slicing

In questa sezione verrà discusso il modo in cui è stato possibile effettuare lo *slicing* della rete al fine di differenziare il trattamento di diversi flussi. In questo caso particolare, sfruttando la stessa vista logica della rete utilizzata nella sezione precedente, facciamo in modo che lo *slice* inferiore sia destinato al traffico normale e alla connessione di tutti gli host tra di loro, mentre lo *slice* superiore sia destinato al traffico video. Per effettuare questa separazione, vengono implementati dei filtri sugli switch s1 ed s2. Gli switch s3 ed s4 controllano che effettivamente il flusso sia di tipo *streaming video*, altrimenti non inoltrano il pacchetto ed interrompono il flusso. Non sono state apportate modifiche alla struttura della rete tramite Mininet, quindi di seguito vengono esposte le modifiche alla classe del controllore Ryu.

```

1 @set_ev_cls(ofp_event.EventOFPSwitchFeatures, CONFIG_DISPATCHER)
2 def switch_features_handler(self, ev):
3     datapath = ev.msg.datapath
4     ofproto = datapath.ofproto
5     parser = datapath.ofproto_parser
6     dpid = datapath.id
7     self.logger.info(f"[FEATURES HANDLER] dpid={dpid}")
8     if dpid == 1:
9         port_up, port_dw = 3, 4
10        port_h1, port_h2 = 1, 2
11        for host_port in [port_h1, port_h2]:
12            match = parser.OFPMatch(
13                in_port=host_port,
14                eth_type=0x0800,
15                ip_proto=17,
16                udp_dst=UDP_PORT_STREAMING
17            )
18            actions = [parser.OFPActionOutput(port_up)]
19            self.add_flow(datapath, priority=100, match=match, actions=actions)
20    elif dpid == 2:
21        port_up, port_dw = 1, 2
22        port_h3, port_h4 = 3, 4
23        for host_port in [port_h3, port_h4]:
24            match = parser.OFPMatch(
25                in_port=host_port,
26                eth_type=0x0800,
27                ip_proto=17,
28                udp_dst=UDP_PORT_STREAMING
29            )
30            actions = [parser.OFPActionOutput(port_up)]
31            self.add_flow(datapath, priority=100, match=match, actions=actions)
32    elif dpid == 3:
33        match = parser.OFPMatch(
34            in_port=1, eth_type=0x0800, ip_proto=17, udp_dst=UDP_PORT_STREAMING
35        )
36        actions = [parser.OFPActionOutput(2)]
37        self.add_flow(datapath, priority=100, match=match, actions=actions)
38        match = parser.OFPMatch(
39            in_port=2, eth_type=0x0800, ip_proto=17, udp_dst=UDP_PORT_STREAMING
40        )
41        actions = [parser.OFPActionOutput(1)]
42        self.add_flow(datapath, priority=100, match=match, actions=actions)
43    elif dpid == 4:
44        match = parser.OFPMatch(in_port=1)
45        actions = [parser.OFPActionOutput(2)]
46        self.add_flow(datapath, priority=1, match=match, actions=actions)
47        match = parser.OFPMatch(in_port=2)
48        actions = [parser.OFPActionOutput(1)]
49        self.add_flow(datapath, priority=1, match=match, actions=actions)
50    # regola di base -> manda pacchetto al controller
51    match = parser.OFPMatch()
52    actions = [parser.OFPActionOutput(ofproto.OFPP_CONTROLLER,
53                                     ofproto.OFPCML_NO_BUFFER)]
54    self.add_flow(datapath, priority=0, match=match, actions=actions)

```

Il service based forwarding è palesato nelle istruzioni che riguardano il match dei pacchetti sugli switch marginali, in cui viene definita la regola per cui se il pacchetto usa il protocollo IP/UDP sul porto 9999 (streaming video) viene mandato sempre sul path superiore, indipendentemente dal MAC dei dispositivi comunicanti. Gli switch s3 ed s4 invece si occupano in questo caso solo del forwarding bidirezionale nel caso in cui avvenga il match (IP/UDP sul porto 9999 per lo switch s3 e tutti gli altri pacchetti per lo switch s4).

```

1 @set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
2 def _packet_in_handler(self, ev):
3     msg = ev.msg
4     datapath = msg.datapath
5     ofproto = datapath.ofproto
6     parser = datapath.ofproto_parser
7     dpid = datapath.id
8     in_port = msg.match['in_port']
9
10    pkt = packet.Packet(msg.data)
11    eth = pkt.get_protocol(ethernet.ethernet)
12    if eth is None:
13        return
14    dst = eth.dst
15    src = eth.src
16    self.mac_to_port.setdefault(dpid, {})
17
18    self.mac_to_port[dpid][src] = in_port
19
20    # dest mac conosciuto
21    if dst in self.mac_to_port[dpid]:
22        out_port = self.mac_to_port[dpid][dst]
23        actions = [parser.OFPActionOutput(out_port)]
24    else:
25        host_ports = {
26            1: {1,2}, # [dpid] -> [h*,h*]
27            2: {3,4},
28            3: set(), # core UP
29            4: set() # core down
30        }
31
32        interswitch_links = {
33            1: {4}, # [dpid] -> down path
34            2: {2},
35        }
36
37        actions = []
38        for p in host_ports.get(dpid, set()):
39            if p != in_port:
40                actions.append(parser.OFPActionOutput(p))
41
42        for p in interswitch_links.get(dpid, set()):
43            if p != in_port:
44                actions.append(parser.OFPActionOutput(p))
45
46    if len(actions) == 1:
47        match = parser.OFPMatch(in_port=in_port, eth_src=src, eth_dst=dst)
48        self.add_flow(datapath, priority=1, match=match, actions=actions)
49
50    out = parser.OFPacketOut(
51        datapath=datapath,
52        buffer_id=msg.buffer_id,
53        in_port=in_port,
54        actions=actions,
55        data=msg.data
56    )
57    datapath.send_msg(out)

```

Questa funzione viene chiamata ogni volta che uno switch riceve un pacchetto, non trova nessun match e inoltra il pacchetto al controller. Il controller estrae informazioni dal pacchetto e mantiene una tabella MAC per ogni switch, dove si trova l'associazione (MAC, port). Se la destinazione è già nota, questa viene utilizzata per l'inoltro diretto. Se la destinazione è ignota, viene inviato il pacchetto in flooding solo sulla rotta down. Il test di questa implementazione è stato effettuato mediante i tool *pingall* di Mininet, *Wireshark* e *iPerf* per la simulazione di traffico.

Command Line

```
mininet> pingall
*** Ping: testing ping reachability

h1 -> h2 h3 h4
h2 -> h1 h3 h4
h3 -> h1 h2 h4
h4 -> h1 h2 h3

*** Results: 0% dropped (12/12 received)
```

La verifica con *iperf* e *wireshark* è stata effettuata configurando h1 come server e h4 come client. Lo scopo era evidenziare attraverso l'ascolto dei link collegati allo switch s3 che:

- le politiche di service slicing sono state effettivamente applicate, infatti il traffico "video streaming" è visto solo sul path superiore e tutto l'altro traffico non è mai visto sul path superiore;
- la banda della rete dedicata è maggiore di quella della rete *best effort* per costruzione, e quindi il traffico video gode di un throughput migliore.

Command Line

```
mininet> h1 iperf -s -u -p 9999
mininet> h4 iperf -c h1 -u -p 9999 -b 10M -t 10
mininet> h1 killall iperf
mininet> h1 iperf -s -u -p 5000
mininet> h4 iperf -c h1 -u -p 5000 -b 1M -t 10
```

[illegible]

10.0.0.1	10.0.0.3	ICMP	98 Echo (ping) request
10.0.0.3	10.0.0.1	ICMP	98 Echo (ping) reply
10.0.0.1	10.0.0.3	ICMP	98 Echo (ping) request
10.0.0.3	10.0.0.1	ICMP	98 Echo (ping) reply
10.0.0.1	10.0.0.4	ICMP	98 Echo (ping) request
10.0.0.4	10.0.0.1	ICMP	98 Echo (ping) reply
10.0.0.2	10.0.0.3	ICMP	98 Echo (ping) request
10.0.0.1	10.0.0.4	ICMP	98 Echo (ping) request

[illegible]

